

DM2026 Assignment 2: Cross-Validation, SVM, Association Rules, and Clustering

Yu-Shun Liu
Student ID: 413551030

GitHub Repository: <https://github.com/RaisoLiu/DM2026-Assignment-2>

1 Q1: K-Fold Cross-Validation

Continuation of Assignment 1 `Real_World_Classification.ipynb`, using the updated `linear_model.py` from the course GitHub. The same preprocessing pipeline is reproduced: `LabelEncoder` on `Species`, `KNNImputer`($k = 5$), train/test split (70/30, seed=40), and `MinMaxScaler` (fit on training data only).

1.1 1(a) 5-Fold Cross-Validation: 4×4 Hyperparameter Grid

We perform 5-fold CV (`KFold(n_splits=5, shuffle=True, random_state=40)`) on the training set (350 samples) over 16 combinations of learning rate and L2 regularization strength. The model uses `n_iteration=10000`, `val_ratio=0.2` (an internal validation split used only for early stopping within each fold; it does not affect the outer CV evaluation, which always scores on the held-out fold).

Table 1: Mean CV accuracy for each (lr, λ) combination

	$\lambda = 1.0$	$\lambda = 2.0$	$\lambda = 4.0$	$\lambda = 8.0$
lr=0.005	0.7200	0.7257	0.7314	0.7257
lr=0.01	0.7257	0.7286	0.7286	0.7229
lr=0.1	0.7171	0.7286	0.7257	0.7229
lr=0.5	0.7171	0.6886	0.5229	0.4657

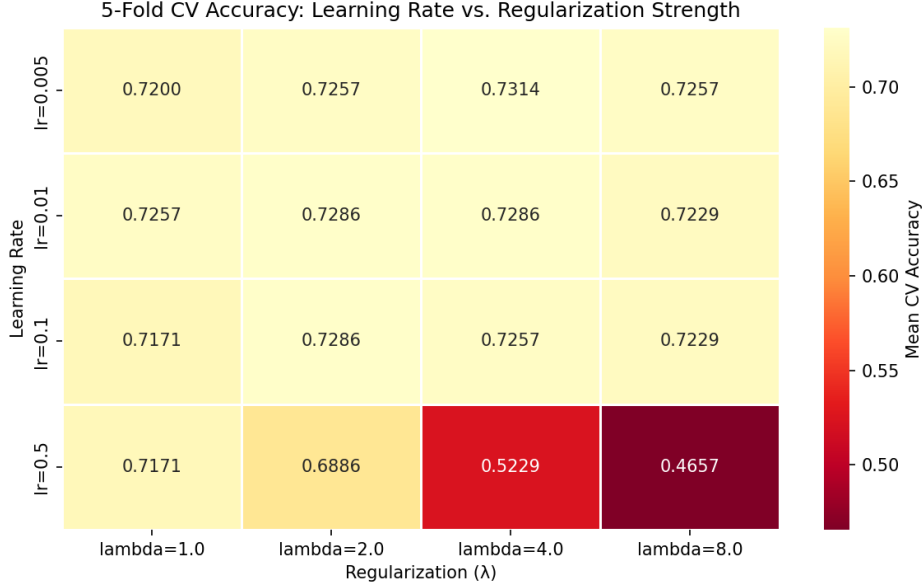


Figure 1: Heatmap of 5-fold CV accuracy across the 4×4 hyperparameter grid. Darker cells indicate lower accuracy; the sharp degradation at $\text{lr}=0.5$ with large λ is immediately visible.

1.2 1(b) Top 2 Hyperparameter Settings — Test Evaluation

The top 2 settings by CV accuracy are retrained on the full training set and evaluated on the held-out test set (150 samples). Note: the second-best CV accuracy (0.7286) is shared by three settings: (0.01, 2.0), (0.01, 4.0), and (0.1, 2.0). We select (0.1, 2.0) as #2 by choosing the setting with the largest learning rate (faster convergence) and smallest λ (least regularization).

Table 2: Test evaluation for top 2 hyperparameter settings

Rank	Setting	Accuracy	Precision	Recall	F1-score
#1	lr=0.005, $\lambda=4.0$	0.7600	0.7273	0.8421	0.7805
#2	lr=0.1, $\lambda=2.0$	0.7467	0.7215	0.7895	0.7540

1.3 1(c) Observations

- **Small learning rates are robust to regularization strength.** Learning rates 0.005, 0.01, and 0.1 achieve similar CV accuracy ($\sim 72\text{--}73\%$) across all λ values, indicating that slow, stable convergence makes the model less sensitive to the regularization hyperparameter.
- **Large learning rate + strong regularization causes severe underfitting.** At $\text{lr}=0.5$, performance degrades sharply when $\lambda \geq 4.0$ (CV accuracy drops to 52.3% and 46.6%). When gradient steps are too large and regularization aggressively shrinks weights, the model cannot learn meaningful patterns.
- **Good generalization from CV to test.** The top-1 setting ($\text{lr}=0.005$, $\lambda=4.0$) achieves 73.1% CV accuracy and 76.0% test accuracy, and the top-2 setting generalizes similarly ($\sim 73\% \rightarrow 74.7\%$). This confirms that 5-fold CV on this dataset provides a reliable estimate of out-of-sample performance.

- The top-1 model shows higher recall (84.2%) than precision (72.7%), meaning it favors identifying positive samples at the cost of more false positives.

2 Q2: SVM Classification

We use the `mobile_price.csv` dataset (2000 samples, 20 features, 4 balanced classes). Data is split 60/20/20 into train/validation/test sets (`random_state=42`). We use `sklearn.svm.SVC` with an RBF kernel and $\gamma = \text{scale}$. This experiment keeps the raw feature representation fixed because it gives strong empirical validation/test performance. However, $\gamma = \text{scale}$ is only a global kernel-coefficient heuristic and does not replace per-feature standardization; standardizing features remains a reasonable additional preprocessing variant. F1-score uses `macro` averaging throughout.

2.1 2(a) SVM with C=1.0

Table 3: SVM (C=1.0) performance across data splits

Set	Accuracy	F1 (macro)
Train	0.9500	0.9505
Validation	0.9475	0.9463
Test	0.9625	0.9617

2.2 2(b) Performance vs. Regularization Parameter C

We sweep $C \in \{0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000\}$.

Table 4: SVM performance for different C values

C	Train Acc	Train F1	Val Acc	Val F1	Test Acc	Test F1
0.001	0.2625	0.1040	0.2325	0.0943	0.2300	0.0935
0.01	0.3242	0.2182	0.3075	0.2249	0.3000	0.2111
0.1	0.8942	0.8959	0.8950	0.8949	0.9000	0.8987
1	0.9500	0.9505	0.9475	0.9463	0.9625	0.9617
10	0.9692	0.9696	0.9450	0.9442	0.9700	0.9693
100	0.9775	0.9778	0.9550	0.9543	0.9700	0.9696
1000	0.9867	0.9868	0.9500	0.9488	0.9700	0.9692
10000	0.9975	0.9975	0.9400	0.9389	0.9700	0.9697

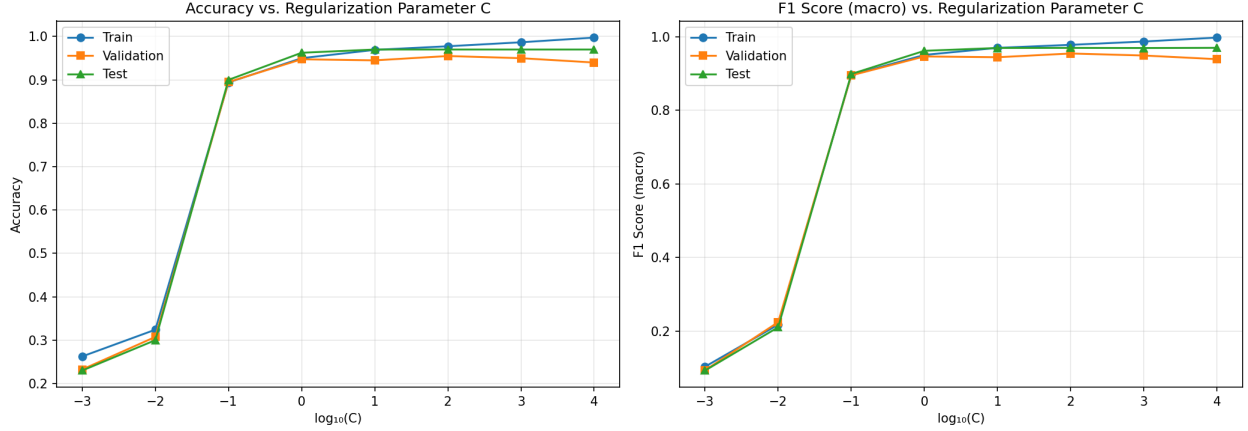


Figure 2: Accuracy and F1 score vs. $\log_{10}(C)$ for train, validation, and test sets.

2.3 2(c) Best Generalization Performance

Best $C = 100$ (selected by highest validation accuracy = 95.5%).

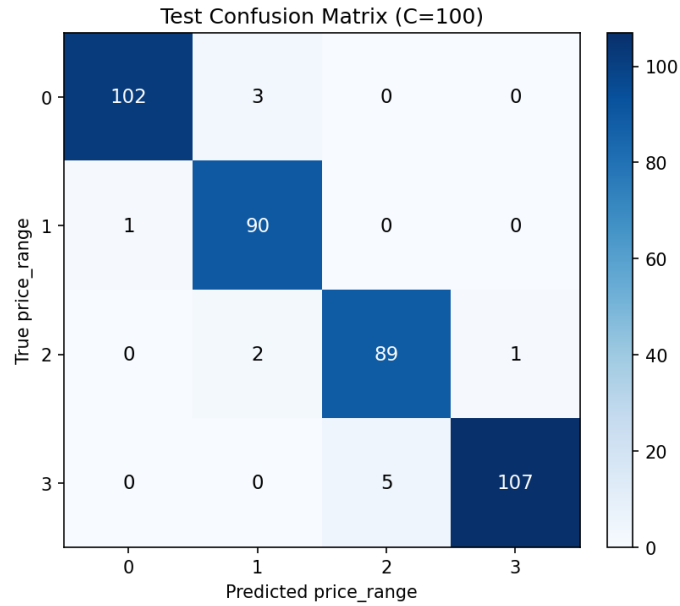


Figure 3: Test confusion matrix for the validation-selected SVM ($C = 100$). Most errors occur between adjacent price ranges, with no extreme confusion between the lowest and highest classes.

- **Underfitting at small C :** With $C \leq 0.01$, the SVM's decision boundary is overly constrained, yielding accuracy near random chance (25% for 4 classes). The margin is too wide and misclassifies most samples.
- **Optimal range:** Performance improves dramatically from $C = 0.1$ ($\sim 89\%$) to $C = 1.0$ ($\sim 95\%$). Beyond $C = 1$, training accuracy continues climbing (reaching 99.8% at $C = 10000$) while validation accuracy peaks at $C = 100$ (95.5%) and then declines — a classic overfitting signal.

- **C=100 balances fit and generalization:** The train-validation gap is moderate (2.25%), and test performance is strong (97.0% accuracy, 97.0% F1). Although $C \in \{10, 1000, 10000\}$ achieve the same test accuracy, $C = 100$ has the best validation score, making it the most reliable choice without peeking at test data.
- **Errors are mostly ordinal-neighbor mistakes:** The confusion matrix shows only 12 test errors out of 400 samples. They occur between neighboring classes (0/1, 1/2, 2/3), which is expected because `price_range` is an ordered label.

3 Q3: Association Rule Mining

We filter samples with `price_range == 1` (500 samples) and select 4 features: `ram`, `int_memory`, `px_width`, `battery_power`. Each feature is discretized into low/medium/high using a 3:4:3 **value range** split:

$$\text{low} \leq \min + 0.3 \times \text{range}, \quad \text{medium} \leq \min + 0.7 \times \text{range}, \quad \text{high} > \min + 0.7 \times \text{range}$$

The discretized features are converted to a boolean one-hot transaction matrix (500×12).

3.1 3(a) Frequent Patterns (support ≥ 0.3)

FP-Growth finds 8 frequent patterns:

Table 5: Frequent itemsets with support ≥ 0.3

Itemset	Support
{ram_medium}	0.682
{px_width_medium}	0.416
{battery_power_medium}	0.414
{int_memory_medium}	0.412
{battery_power_medium, ram_medium}	0.318
{int_memory_low}	0.316
{battery_power_low}	0.308
{ram_medium, px_width_medium}	0.306

3.2 3(b) Association Rules (support ≥ 0.3 , confidence ≥ 0.4 , lift ≥ 0.8)

Table 6: Association rules meeting all criteria

Antecedent	Consequent	Support	Confidence	Lift
ram_medium	battery_power_medium	0.318	0.466	1.126
battery_power_medium	ram_medium	0.318	0.768	1.126
ram_medium	px_width_medium	0.306	0.449	1.079
px_width_medium	ram_medium	0.306	0.736	1.079

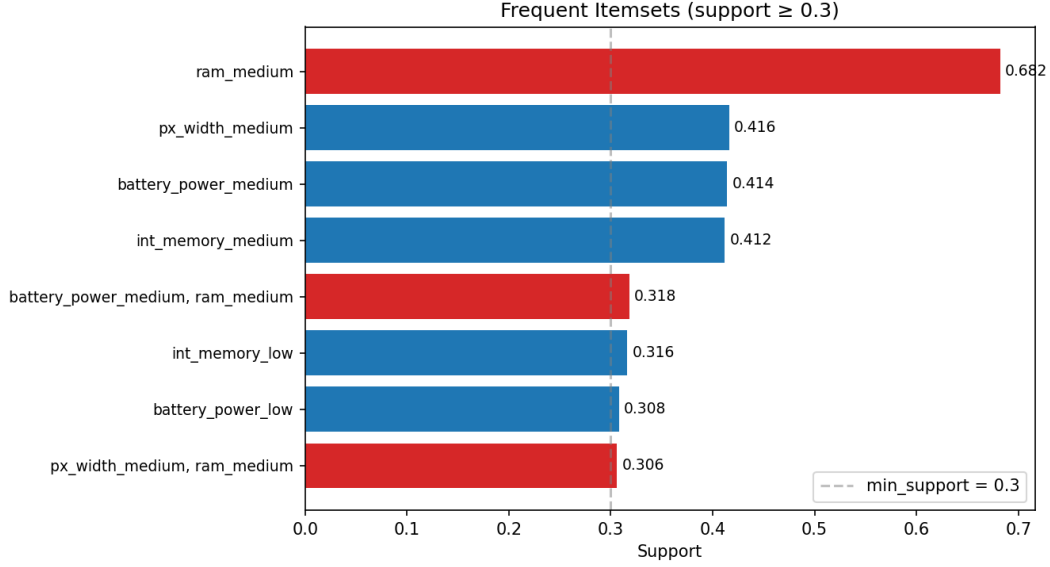


Figure 4: Support of frequent itemsets. Items involving **ram_medium** (red) dominate.

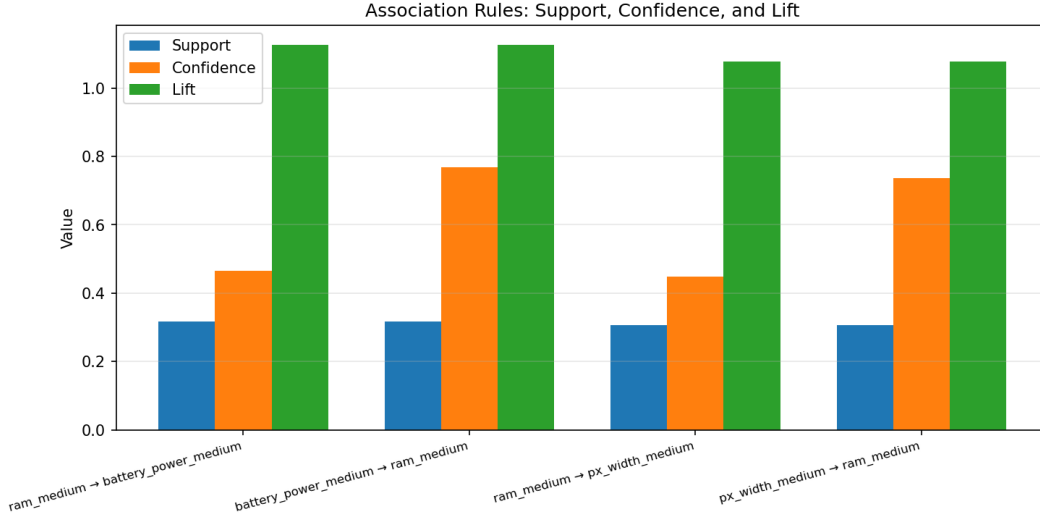


Figure 5: Support, confidence, and lift for each association rule. The asymmetric confidence between forward and reverse rules is clearly visible.

3.3 3(c) Observations

- **ram_medium dominates:** 68.2% of price_range=1 phones have mid-range RAM, making it the most frequent single item and the anchor of both 2-item frequent patterns. This aligns with the second-lowest price tier having predominantly mid-range specifications.
- **Asymmetric confidence:** `battery_power_medium → ram_medium` has 76.8% confidence, while the reverse has only 46.6%. This asymmetry arises because `ram_medium` has much higher base support (68.2%), so knowing a phone has medium battery power strongly predicts medium RAM, but not vice versa.

- **Mild positive associations:** All lift values are close to 1.0 (1.08–1.13), indicating that while medium-range features tend to co-occur slightly more than expected by chance, the associations are not dramatically strong. Within this price tier, the four features are somewhat independent.
- Under the assignment thresholds (support ≥ 0.3 , confidence ≥ 0.4 , lift ≥ 0.8), only 2 two-item patterns and 4 rules are retained. This is consistent with the relatively concentrated mid-range specifications in this price tier, though the high support threshold also limits the number of discoverable patterns.

4 Q4: PCA and K-Means Clustering

All 20 features from `mobile_price.csv` are standardized using z-score (`StandardScaler`). The target variable `price_range` (4 classes) is used only for evaluation, not for clustering.

4.1 4(a) Standardization

After `StandardScaler`, all features have mean ≈ 0 and std = 1.

4.2 4(b) PCA 2D Projection with True Labels

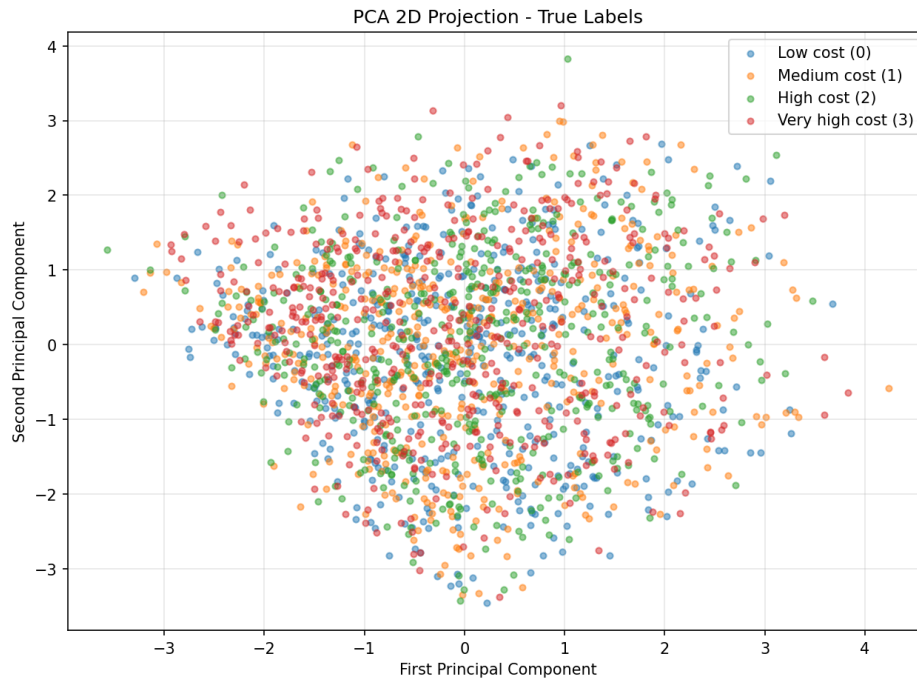


Figure 6: PCA 2D projection colored by true price-range labels. The first two PCs explain only 16.5% of total variance. Classes overlap heavily in this projection.

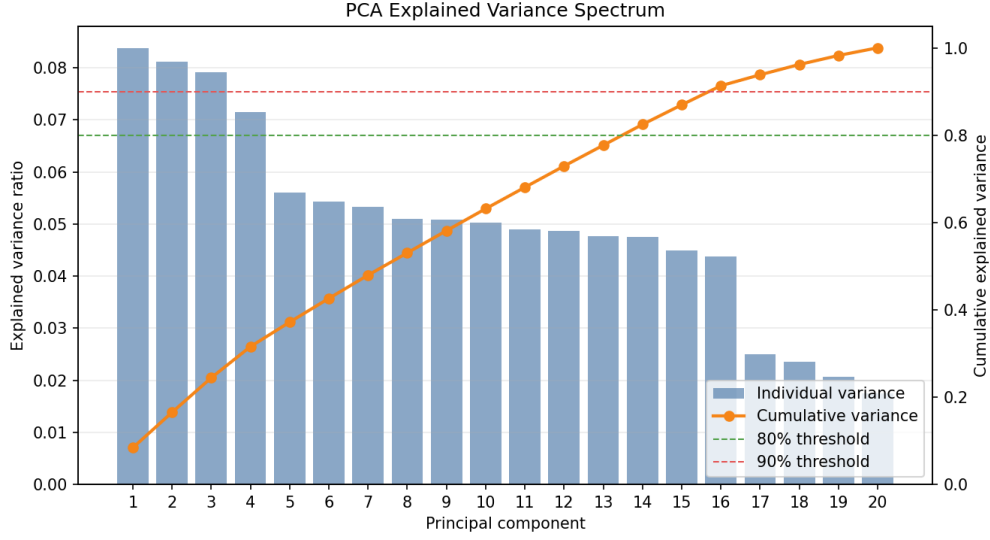


Figure 7: PCA explained-variance spectrum. The first 2 PCs explain 16.5% of variance, the first 5 explain 37.2%, and the first 10 explain 63.1%; 14 PCs are needed to reach 80% and 16 PCs are needed to reach 90%.

4.3 4(c) K-Means on All Standardized Features

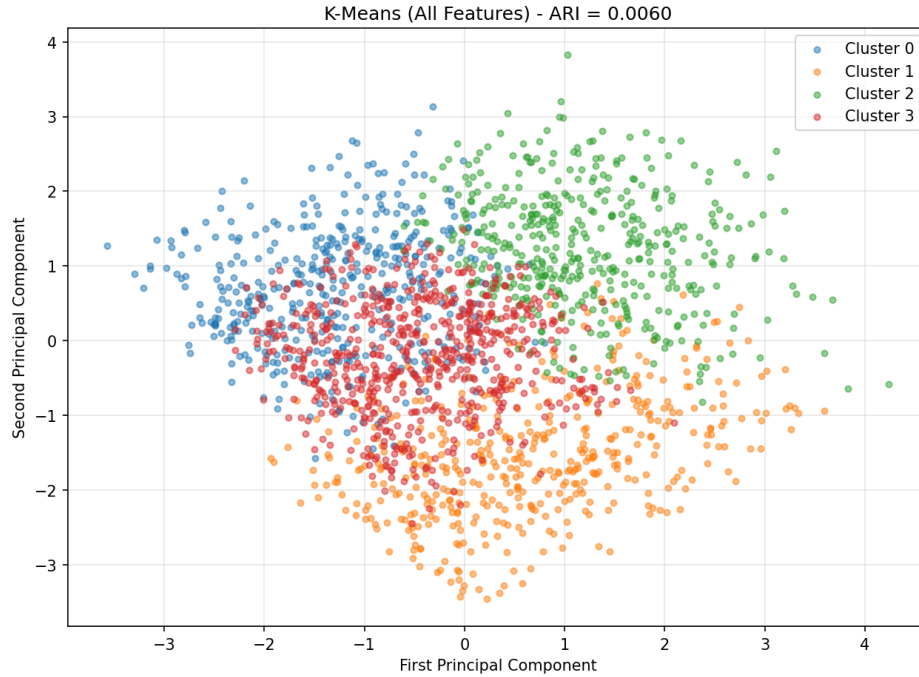


Figure 8: K-Means (all 20 features) visualized on PCA 2D plane. ARI = 0.0060.

4.4 4(d) K-Means on PCA 2D Features

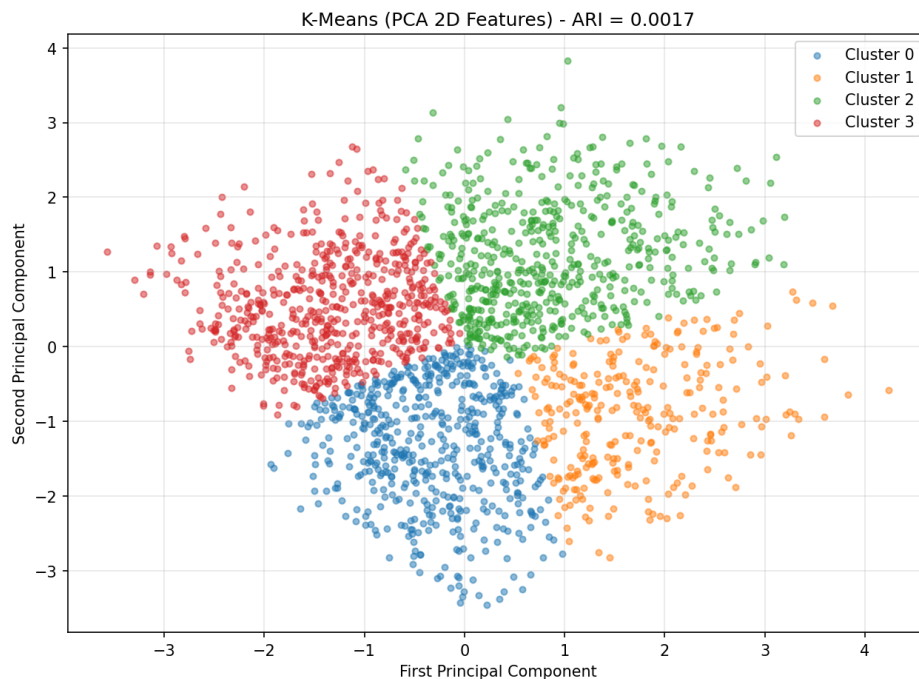


Figure 9: K-Means on PCA 2D features. ARI = 0.0017.

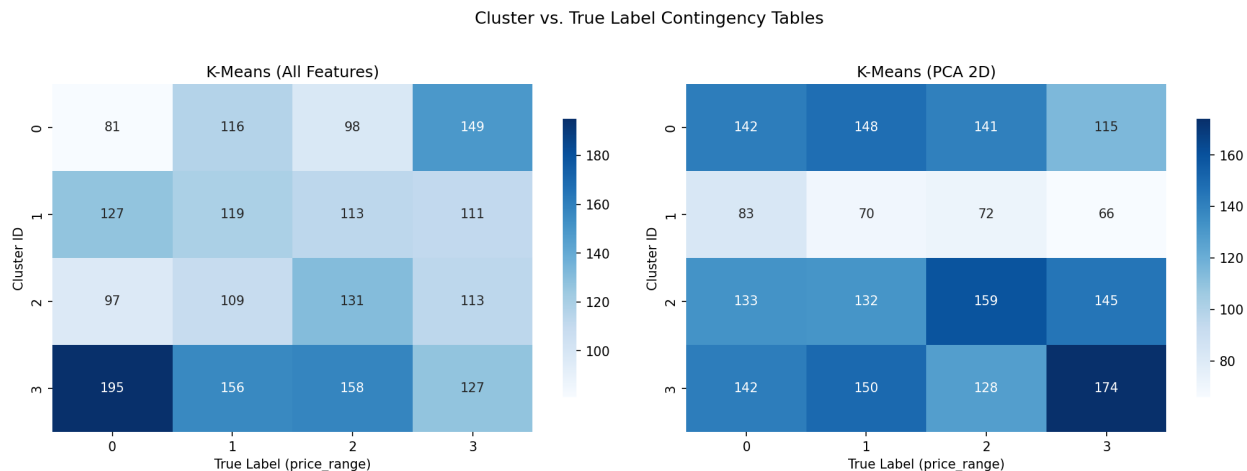


Figure 10: Contingency tables: cluster assignment vs. true label. Both heatmaps show diffuse distributions across all classes, confirming that K-Means clusters do not align with price-range labels.

4.5 4(e) Observations

- PCA captures very little variance in two dimensions:** The first two principal components explain only 16.5% of total variance. The full spectrum (Figure 7) confirms that the information is broadly distributed: even 10 PCs retain only 63.1% variance, and 14 PCs

are needed to reach 80%. Therefore the 2D projection is useful for visualization but too compressed to preserve the full structure.

- **Both K-Means approaches fail:** ARI values of 0.0060 (all features) and 0.0017 (PCA 2D) are near zero, meaning the clustering is essentially random with respect to the true labels.
- **Dimensionality reduction hurts slightly:** K-Means on all 20 features (ARI=0.0060) marginally outperforms K-Means on 2D PCA (ARI=0.0017), likely consistent with the information loss from reducing to 2D (which retains only 16.5% of total variance), though PCA variance retention and class separability are not the same thing.
- **K-Means fails to recover label-consistent partitions:** K-Means assumes spherical, equally-sized clusters in Euclidean space, which does not match the structure of this dataset. While supervised SVM (Q2) achieves >95% accuracy by exploiting label information to learn non-linear decision boundaries, K-Means — as an unsupervised method — cannot leverage labels and relies solely on Euclidean distance, which is insufficient to separate these price-range classes. This is not an apples-to-apples comparison, but it highlights that label information is critical for this classification task.

5 Q5: Enhancing K-Means with Association Rule Mining

5.1 Motivation and Inspiration

Standard K-Means treats all features equally and cannot distinguish discriminative features from noise. We propose a **semi-supervised** enhancement that uses association rule mining to guide clustering. The approach explicitly leverages class labels during feature engineering, while K-Means itself remains unsupervised.

Inspirations:

- *TF-IDF-style feature importance:* We adapt the idea of term specificity (IDF) to use association rule lift as a proxy for discriminative power. Features that appear in high-lift rules for specific classes are analogous to terms with high IDF — they carry more information.
- *Supervised discretization* (Fayyad & Irani, 1993): Class labels guide the mining process to identify feature patterns that maximize class separability, similar to how supervised binning optimizes split points.
- *Feature augmentation for clustering:* Enriching the feature space with auxiliary pattern-based features before applying standard clustering, as discussed in lecture on pattern-based clustering.

5.2 Method Design

We design two components and evaluate them both independently and jointly via ablation:

1. **Rule-Guided Feature Weighting (primary method, semi-supervised):** All 20 features are discretized (3:4:3 value range split) and association rules are mined *per class* using FP-Growth (`min_support=0.3`, `confidence ≥ 0.5`, `lift ≥ 1.0`). For each original feature, we accumulate the lift values from all rules in which the feature appears across all classes. The resulting importance scores are normalized to weights in $[1.0, 3.0]$ and applied to the standardized features. This step is explicitly label-guided.

2. **Class-Stratified Pattern Features (experimental extension, semi-supervised):** The top 5 most discriminative patterns (ranked by lift $\times$ confidence) are selected from each class’s rules, yielding 20 binary features indicating whether a sample matches each class-associated itemset.

As a control experiment, we also evaluate a **label-free global rule weighting** variant. It mines FP-Growth rules on all samples without using `price_range`, converts global rule lift values into feature weights using the same normalization procedure, and then runs K-Means. This sanity check tests whether the improvement comes from association-rule weighting in general or specifically from the label-guided rule-mining step.

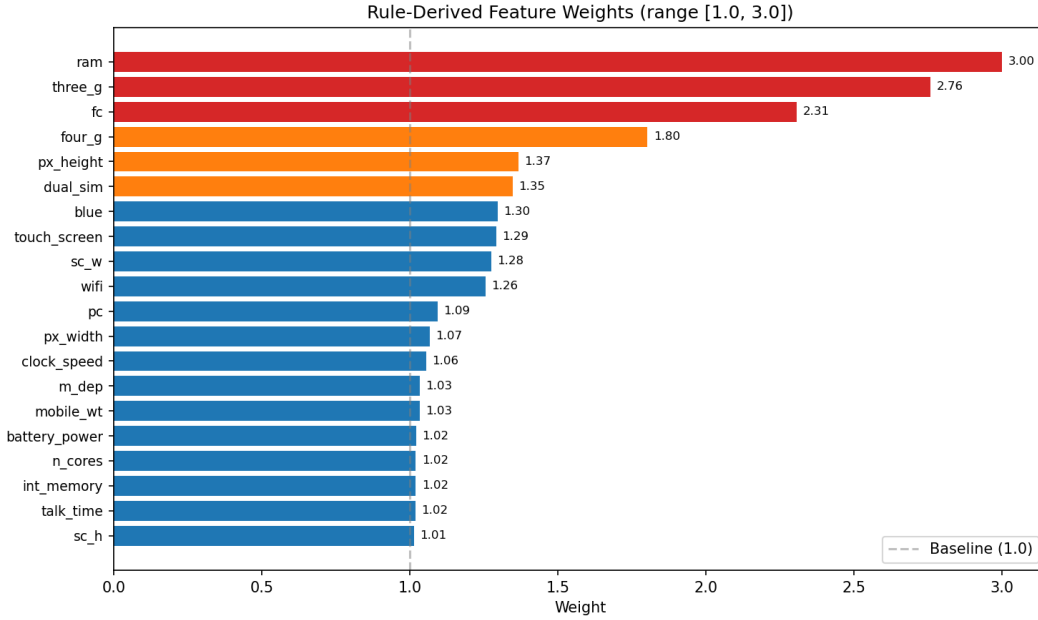


Figure 11: Rule-derived feature weights. `ram` (3.00), `three_g` (2.76), and `fc` (2.31) receive the highest weights, indicating they appear most frequently in high-lift class-specific rules.

5.3 Evaluation Protocol

All variants are evaluated using the same protocol:

- 5 random seeds: {0, 10, 42, 100, 999}
- **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`) maps cluster IDs to true labels optimally before computing accuracy, precision, recall, and F1 (all macro-averaged).
- **Clustering-native metrics:** ARI (Adjusted Rand Index) and NMI (Normalized Mutual Information) are also reported, which do not require Hungarian mapping.

5.4 Ablation Study

To isolate the contribution of each component and test the source of the improvement, we compare four method variants plus one label-free control:

Table 7: Ablation study and sanity check: average performance across 5 seeds

Method	Accuracy	Precision	Recall	F1	ARI	NMI
Baseline K-Means	0.2962	0.2959	0.2962	0.2941	0.0058	0.0078
Global-rule weighting	0.2660	0.2685	0.2660	0.2606	0.0008	0.0025
Weighting-only	0.4421	0.4250	0.4421	0.4285	0.1769	0.2109
Pattern-only	0.2620	0.2632	0.2620	0.2539	−0.0005	0.0013
Full Method	0.3889	0.4300	0.3889	0.3961	0.1009	0.1518

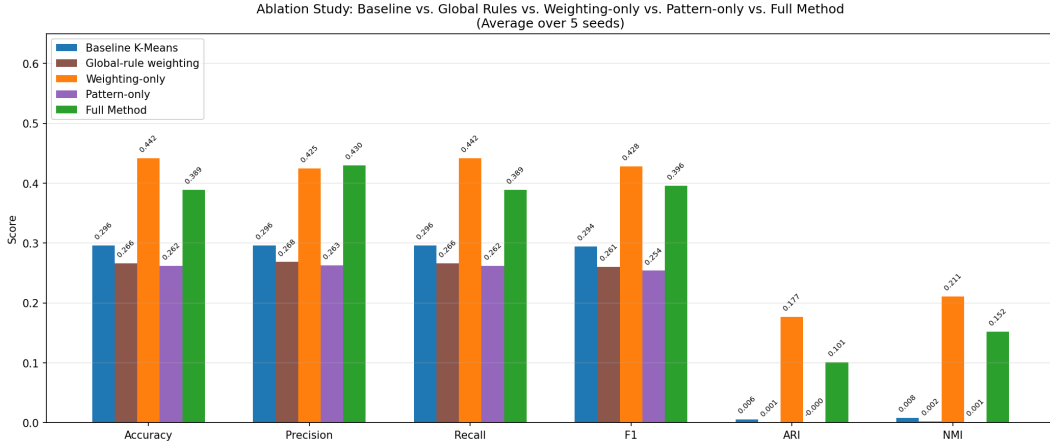


Figure 12: Ablation study and label-free sanity check: average performance comparison across 5 seeds.

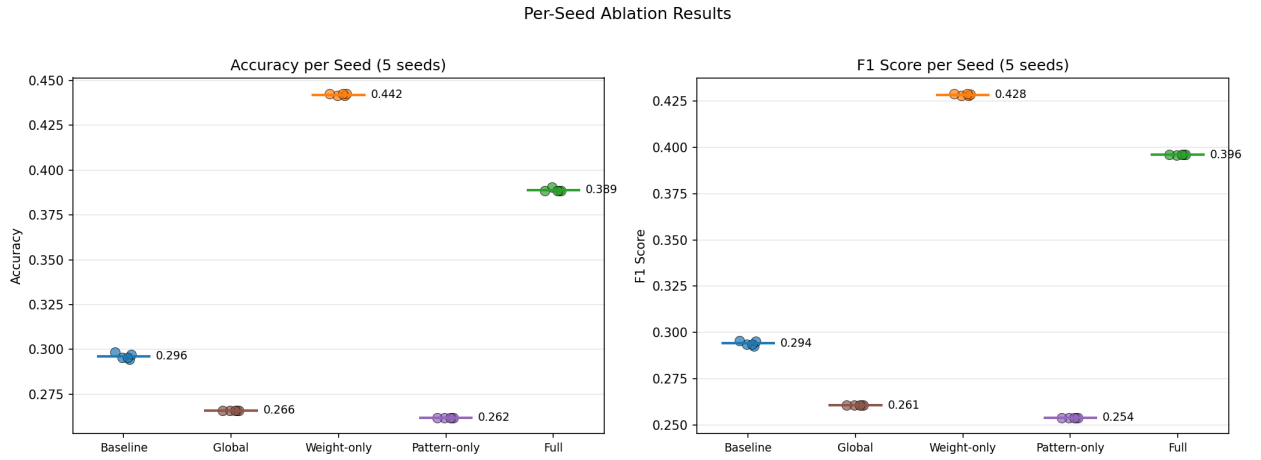


Figure 13: Per-seed accuracy and F1 scores. Weighting-only is consistently the best across all seeds; pattern-only is consistently the worst. Horizontal lines indicate means.

5.5 Analysis and Recommended Method

The ablation study reveals a clear winner: **weighting-only is the recommended enhanced method.**

Table 8: Final comparison: Baseline K-Means vs. Recommended Method (Weighting-only)

Method	Accuracy	Precision	Recall	F1	ARI	NMI
Baseline K-Means	0.2962	0.2959	0.2962	0.2941	0.0058	0.0078
Weighting-only (recommended)	0.4421	0.4250	0.4421	0.4285	0.1769	0.2109
Improvement	+14.6 pp	+12.9 pp	+14.6 pp	+13.4 pp	+0.171	+0.203

- **Feature weighting is the dominant component.** The weighting-only variant achieves the best accuracy (44.2%) and ARI (0.177), outperforming the full method (38.9%, ARI=0.101) and baseline (29.6%, ARI=0.006). The key discriminative features — **ram** (weight=3.0), **three_g** (2.76), and **fc** (2.31) — are upweighted by the rule-derived importance scores, providing a +14.6 pp accuracy improvement over baseline with no additional feature dimensions. This approach is best understood as a semi-supervised enhancement of K-Means: the labels guide the rule-mining step (feature engineering), while the clustering step itself remains purely unsupervised.
- **Label-free rule weighting is a failed sanity check.** Global rule weighting mines 72 frequent itemsets and 39 rules without labels, but reaches only 26.6% accuracy and ARI=0.0008, below the original K-Means baseline. This shows that the gain is not caused by arbitrary association-rule weighting; it depends on class-stratified, label-guided rule mining. Therefore, the proposed method should be interpreted as a semi-supervised enhancement of K-Means rather than a purely unsupervised clustering method.
- **Pattern features alone hurt performance (negative finding).** The pattern-only variant (26.2%) performs *worse* than baseline (29.6%), with ARI near zero. Appending 20 binary pattern features to the 20 original features introduces noise that dilutes the continuous feature signal. This is a valuable negative finding: **more features is not always better**, even when those features encode class-related information.
- **The full method improves over baseline but is not the best variant.** Combining weighting and patterns (38.9%) substantially improves over baseline (29.6%), yet falls short of weighting-only (44.2%) on accuracy, recall, F1, ARI, and NMI. The pattern features partially counteract the benefit of feature weighting. This confirms that the pattern feature component should be discarded in the final method.
- **Clustering-native metrics confirm the ranking.** ARI and NMI follow the same overall ranking as classification metrics (weighting-only > full > baseline > global-rule weighting $\approx$ pattern-only), providing evidence that the cluster *structure itself* improves, not just the post-hoc label mapping. The per-seed plots (Figure 13) show this ranking is consistent across all 5 random seeds.
- **Limitations.** Even the best variant (44.2%) remains far below supervised SVM (97%). K-Means cannot capture non-linear decision boundaries regardless of feature engineering. The semi-supervised nature of this approach (using labels for rule mining) is also a limitation for purely unsupervised settings.